BORDEAUX

ynov

CAMPUS

Manipulation de données en Python

BAPTISTE MOSER - INFO B2-

ANNÉE SCOLAIRE 2024/2025- MARS2025

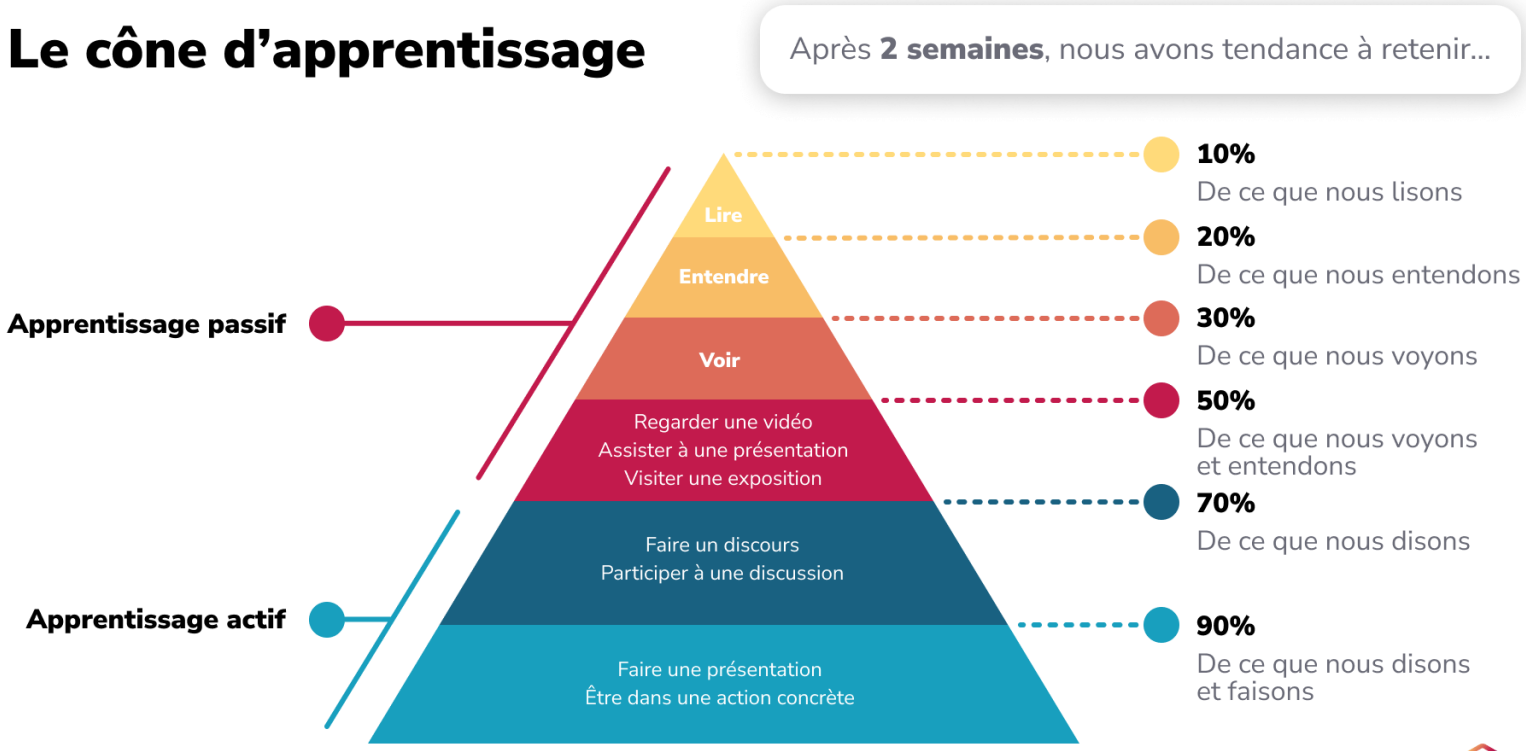
Objectifs généraux

- Comprendre les fondamentaux de la manipulation de données en Python.
- Apprendre à utiliser les bibliothèques clés : pandas, numpy, matplotlib ou seaborn
- Savoir effectuer des opérations de nettoyage, transformation, visualisation et analyse de données.
- Introduire des cas concrets et des projets pour ancrer les compétences.

Approche pédagogique

- Equilibre entre théorie et pratique : pour chaque nouveau concept, nous aurons une partie théorique suivie immédiatement de la partie pratique
- Pourquoi la pratique ?
  - Car la manipulation de données demande des compétences qui se perfectionnent en expérimentant et en résolvant des cas réels (et surtout en se trompant !)
  - Les exercices pratiques permettent de mieux assimiler les outils et d'acquérir les réflexes indispensables pour travailler efficacement dans des milieux professionnels

L'importance de la pratique (apprentissage actif)



Evaluation des compétences

- Les Formats :
  - QCM pour tester les compétences théoriques à plusieurs points clés de notre parcours
  - Projets pratiques pour résoudre des problématiques : permet d'évaluer la capacité à appliquer les compétences en situation "réelle" sur des projets concrets

Evaluation des compétences

Les évaluations	Coefficients
QCM 1 (20 minutes) - 21/03/2025	0.25
QCM 2 (20 minutes) - 04/04/2025	0.25
Projet (1h20 minutes) - 18/04/2025	0.5

Module 1 : Introduction et fondamentaux sur les données

1. Qu’est-ce qu’une donnée ?

2. Pourquoi les données sont-elles si importantes aujourd’hui ?

3. De la donnée brute à la valeur ajoutée

Qu'est-ce qu'une donnée ?

- "La représentation d'une information"
- Plusieurs formats de données : textuelles, numériques, images, sons...
- Plusieurs formats des fichiers

L'importance des données

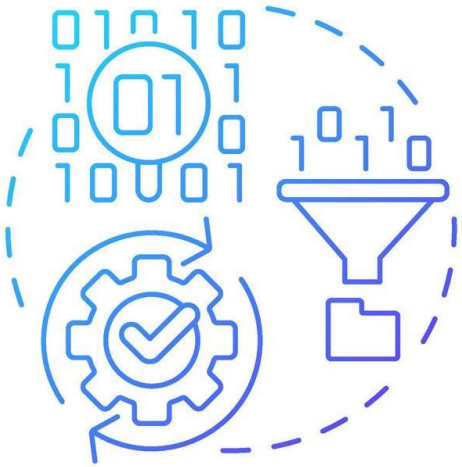
- Aide à la décision : Optimiser les performances, identifier les opportunités, prendre des décisions stratégiques en temps réel

- Avancées technologiques et innovation : IA/Big Data

De la donnée brute à la valeur ajoutée

Concept de manipulation de données

- Chargement
- Exploration
- Nettoyage
- Transformation
- Filtrage et sélection
- Agrégation et regroupement
- Enrichissement



Détail des étapes de la manipulation de données

- Collecte des données : rassembler les données provenant de différentes sources (fichiers csv, bases de données, API)
- Nettoyage des données : élimination des valeurs manquantes ou incohérentes
- Transformation des données : Normaliser, agréger, ou convertir les données selon le besoin (ex: changement de format ou retypage de variables)
- Exploration des données : techniques statistiques pour comprendre les tendances et les relations
- Visualisation : Graphiques, tableaux, communication des résultats

Autres outils utilisés pour manipuler des données

- Excel : simple et efficace pour des manipulations de données légères
- R : Utilisé pour l'analyse statistique
- SQL : langage de requête pour interagir avec les bases de données relationnelles
- Tableau / Power BI : centrés sur la visualisation de données

Exemples d'applications pratiques

- Analyses financières : prévoir les tendances économiques
- E-commerce : analyser le comportement des clients pour la recommandation de produits
- Santé : Analyser des données médicales pour détecter des tendances de santé ou émettre un diagnostic médical
- Sport : Analyser des données de performances afin d'optimiser les entrainements et prévenir les blessures
- Plus globalement, les exemples d'applications sont infinis

Cas réel : Réduction du gaspillage alimentaire chez Carrefour

- **Problème :**
  - Carrefour devait lutter contre le **gaspillage alimentaire** en améliorant la gestion des stocks de produits frais.
  - Les invendus et les produits périmés représentaient une **perte financière** et un **impact environnemental négatif**.

Cas réel : Réduction du gaspillage alimentaire chez Carrefour

- **Solution avec Python :**
  - Carrefour a mis en place un **algorithme prédictif basé sur Python** en utilisant **pandas, NumPy et scikit-learn** pour :
    - **Prédire la demande des produits frais** en fonction des ventes passées, de la météo et des événements locaux.
    - **Optimiser l'approvisionnement** pour éviter les surplus inutiles.
    - **Automatiser les réductions de prix dynamiques** en fonction de la date de péremption, afin d'encourager l'achat avant que les produits ne soient jetés.

Cas réel : Réduction du gaspillage alimentaire chez Carrefour

- **Résultats :**
  - **Réduction du gaspillage alimentaire de 30 %** dans les magasins test.
  - **Optimisation des commandes** pour éviter les stocks excédentaires.
  - **Augmentation des ventes de produits frais** grâce aux remises intelligentes.

Ce projet montre comment **l'analyse des données avec Python peut avoir un impact positif sur l'environnement et l'économie** en réduisant le gaspillage alimentaire.

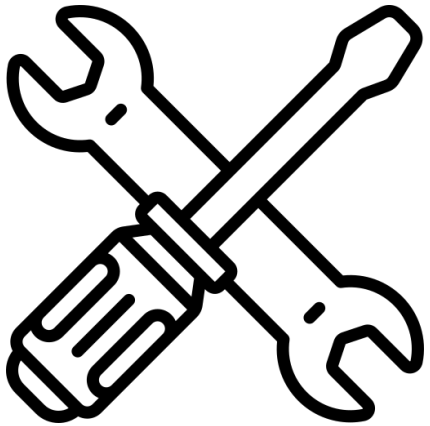
🥦📊🌍

Module 2 : Les différents outils

1. Quels sont les outils ?

2. Quels sont leurs spécificités ?

3. Comment les manipuler ?



Module 2 : Architecture Technique pour ce cours

- Langage : Python
- Bibliothèques : Pandas, Numpy, MatplotLib, Seaborn…
- Plateforme : Anaconda

Python : un peu d'histoire

- Inventeur : Guido van Rossum
- Année de première version : 1991
- Origine du nom : Monty Python

Python : caractéristiques du langage

- Langage de haut niveau
- Langage interprété
- Orienté objet
- Typage dynamique
- Version actuelle : Python 3

Conçu pour être simple, efficace et puissant, ce qui explique son adoption massive dans divers domaines, de l'automatisation à l'intelligence artificielle

Pourquoi choisir Python ?

- Langage de référence pour la manipulation, l'analyse et la visualisation de données
- Simplicité et lisibilité du code
- De nombreuses bibliothèques
- Communauté active et documentation abondante
- Adapté à des projets de toutes tailles

Classement des langages les plus populaires

Jun 2024	Jun 2023	Change	Programming Language	Ratings	Change
1	1		Python	15.39%	+2.93%
2	3	^	C++	10.03%	-1.33%
3	2	v	C	9.23%	-3.14%
4	4		Java	8.40%	-2.88%
5	5		C#	6.65%	-0.06%
6	7	^	JavaScript	3.32%	+0.51%
7	14	^^	Go	1.93%	+0.93%
8	9	^	SQL	1.75%	+0.28%
9	6	v	Visual Basic	1.66%	-1.67%
10	15	^^	Fortran	1.53%	+0.53%
11	11		Delphi/Object Pascal	1.52%	+0.27%
12	19	^^	Swift	1.27%	+0.33%
13	10	v	Assembly language	1.26%	-0.03%
14	12	v	MATLAB	1.26%	+0.14%
15	8	vv	PHP	1.22%	-0.52%
16	13	v	Scratch	1.17%	+0.15%
17	20	^	Rust	1.17%	+0.26%
18	18		Ruby	1.11%	+0.17%
19	29	^^	Kotlin	1.01%	+0.50%
20	22	^	COBOL	0.96%	+0.22%

Les bibliothèques de manipulation de données

- Pandas : Manipulation de données en mémoire pour des projets de taille modeste à moyenne
- Numpy : Calculs numériques rapides et avancés avec des données multidimensionnelles
- PySpark : Traitement des données à grande échelle sur des clusters distribués pour le Big Data

Les bibliothèques de visualisation de données

- MatplotLib : Création de graphiques statiques et interactifs personnalisés

- Seaborn : Simplifier la création de visualisations statistiques et de graphiques plus esthétiques

Les bibliothèques de traitement d'images

- Pourquoi traiter les images ?
  - Améliorer la qualité des images (filtrage, correction des couleurs...)
  - Analyse et reconnaissance (vision par ordinateur, détection d'objets...)
  - Automatisation des tâches graphiques (redimensionnement, conversion de format...)

Pourquoi utiliser des bibliothèques Python pour ces traitements ?

- Simplifier la manipulation des images
- Fournir des algorithmes optimisés et rapides
- Compatibilité avec d'autres outils (IA notamment)

Principales bibliothèques de manipulation d'images en Python

- Pillow : Manipulation de base (ouverture, redimensionnement, filtres...)
- OpenCV : Traitement avancé (détection d'objets, reconnaissance faciale...)
- Numpy : représentation d'images sous un format tabulaire pour des manipulations avancées
- Matplotlib : affichage et annotations sur des images
- Scikit-Image : Traitement scientifique (segmentation, analyse de texture...)
- Imageio : Lecture et écriture d'images sous différents formats (GIF, vidéos...)

Présentation de la bibliothèque NumPy

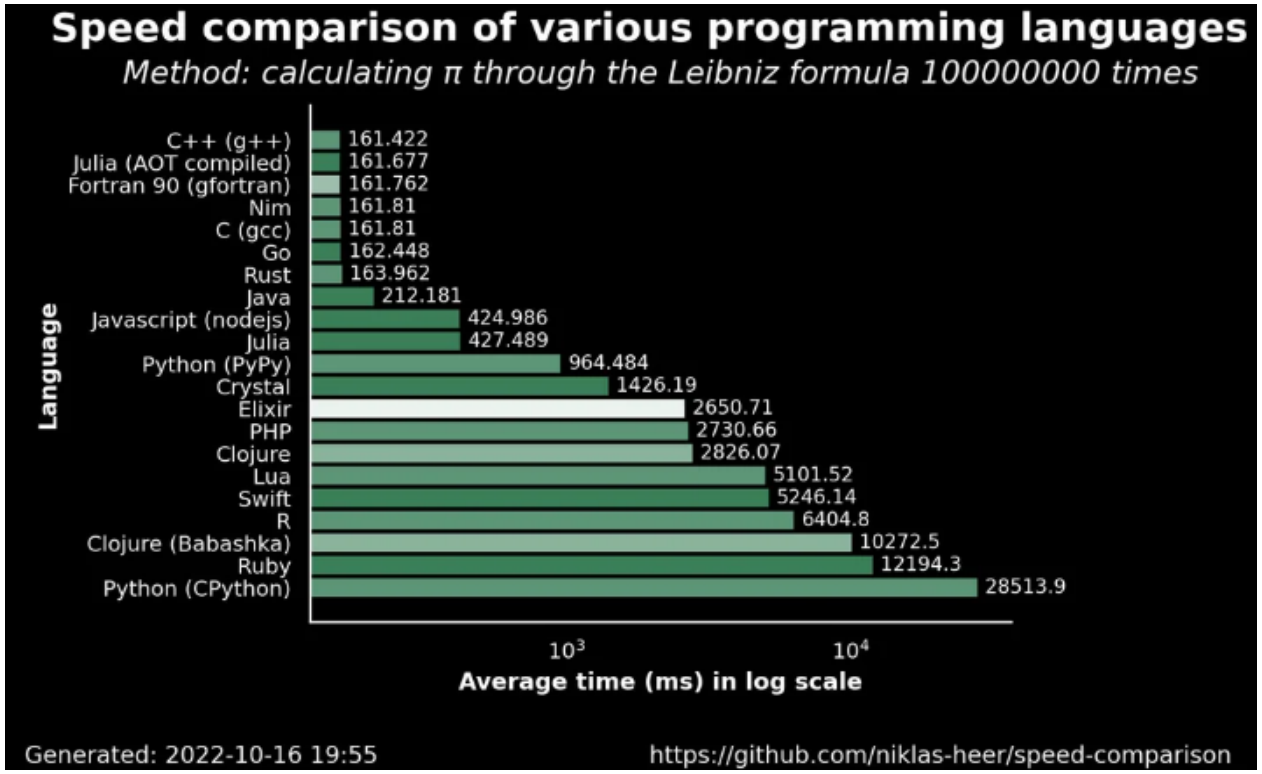
- Créée en 2005 par Travis Oliphant (basée sur Numeric, créée en 1995)
- Bibliothèque Python, avec des parties en C pour optimiser la performance
- But : Calcul numérique rapide avec des tableaux multidimensionnels
- Applications : Science des données, simulation, machine learning, traitement d'image.

Points forts de la bibliothèque NumPy

- Opérations vectorisées ultra-rapides
- Les fonctions mathématiques avancées (algèbre linéaire, transformées de Fourier)
- Large écosystème scientifique (base pour Pandas, SciPy, etc.)

Vitesse d'exécution de différents langages

Total					
	Energy		Time		Mb
(c) C	1.00	(c) C	1.00	(c) Pascal	1.00
(c) Rust	1.03	(c) Rust	1.04	(c) Go	1.05
(c) C++	1.34	(c) C++	1.56	(c) C	1.17
(c) Ada	1.70	(c) Ada	1.85	(c) Fortran	1.24
(v) Java	1.98	(v) Java	1.89	(c) C++	1.34
(c) Pascal	2.14	(c) Chapel	2.14	(c) Ada	1.47
(c) Chapel	2.18	(c) Go	2.83	(c) Rust	1.54
(v) Lisp	2.27	(c) Pascal	3.02	(v) Lisp	1.92
(c) Ocaml	2.40	(c) Ocaml	3.09	(c) Haskell	2.45
(c) Fortran	2.52	(v) C#	3.14	(i) PHP	2.57
(c) Swift	2.79	(v) Lisp	3.40	(c) Swift	2.71
(c) Haskell	3.10	(c) Haskell	3.55	(i) Python	2.80
(v) C#	3.14	(c) Swift	4.20	(c) Ocaml	2.82
(c) Go	3.23	(c) Fortran	4.20	(v) C#	2.85
(i) Dart	3.83	(v) F#	6.30	(i) Hack	3.34
(v) F#	4.13	(i) JavaScript	6.52	(v) Racket	3.52
(i) JavaScript	4.45	(i) Dart	6.67	(i) Ruby	3.97
(v) Racket	7.91	(v) Racket	11.27	(c) Chapel	4.00
(i) TypeScript	21.50	(i) Hack	26.99	(v) F#	4.25
(i) Hack	24.02	(i) PHP	27.64	(i) JavaScript	4.59
(i) PHP	29.30	(v) Erlang	36.71	(i) TypeScript	4.69
(v) Erlang	42.23	(i) Jruby	43.44	(v) Java	6.01
(i) Lua	45.98	(i) TypeScript	46.20	(i) Perl	6.62
(i) Jruby	46.54	(i) Ruby	59.34	(i) Lua	6.72
(i) Ruby	69.91	(i) Perl	65.79	(v) Erlang	7.20
(i) Python	75.88	(i) Python	71.90	(i) Dart	8.64
(i) Perl	79.58	(i) Lua	82.91	(i) Jruby	19.84



Présentation de la bibliothèque Pandas

- Créée en 2008 par Wes McKinney
- Bibliothèque Python, basée sur NumPy
- But : manipulation et analyse de données tabulaires
- Applications : Science des données, finance, statistiques.

Points forts de pandas

- Simple et puissant pour manipuler des données massives
- Supporte de nombreux formats (csv, Excel, SQL, JSON…)
- Communauté active et documentation riche

Présentation de la bibliothèque PySpark

- Créée en 2013 par Matei Zaharia
- But : Manipuler et analyser des données massives avec des fonctionnalités distribuées.
- Applications : Big Data, Machine Learning, ETL, analyse en temps réel

Points forts de PySpark

- Traitement rapide grâce à l'exécution en mémoire
- API Python adaptée aux développeurs Python habitués à Pandas
- Scalabilité (permet de traiter plusieurs Giga sur un seul PC ou de passer des pétaoctets sur des clusters de milliers de machines)

La Plateforme Anaconda

- Créée en 2012 par Travis Oliphant et Peter Wang
- Langue : Python (et R pour certaines extensions)
- But simplifier la gestion des environnements, bibliothèques et workflows en science des données
- Plus de 35 millions d'utilisateurs en 2023
- Applications : Science des données, machine learning, analyse statistique, développement.
- Caractéristiques : Intègre plus de 1500 bibliothèques, des IDE, des gestions d'environnements, gestionnaire de paquets...

Points forts de la plateforme Anaconda

- Facilité d'installation et d'utilisation
- Communauté active et support commercial
- Compatible avec Windows, macOS et Linux

Anaconda : présentation de la plateforme pour notre initiation à la manipulation de données

- Action à réaliser :
  - Téléchargement d'Anaconda : https://www.anaconda.com/download

- Découvrons ensemble la plateforme :

  - les différents logiciels intégrés

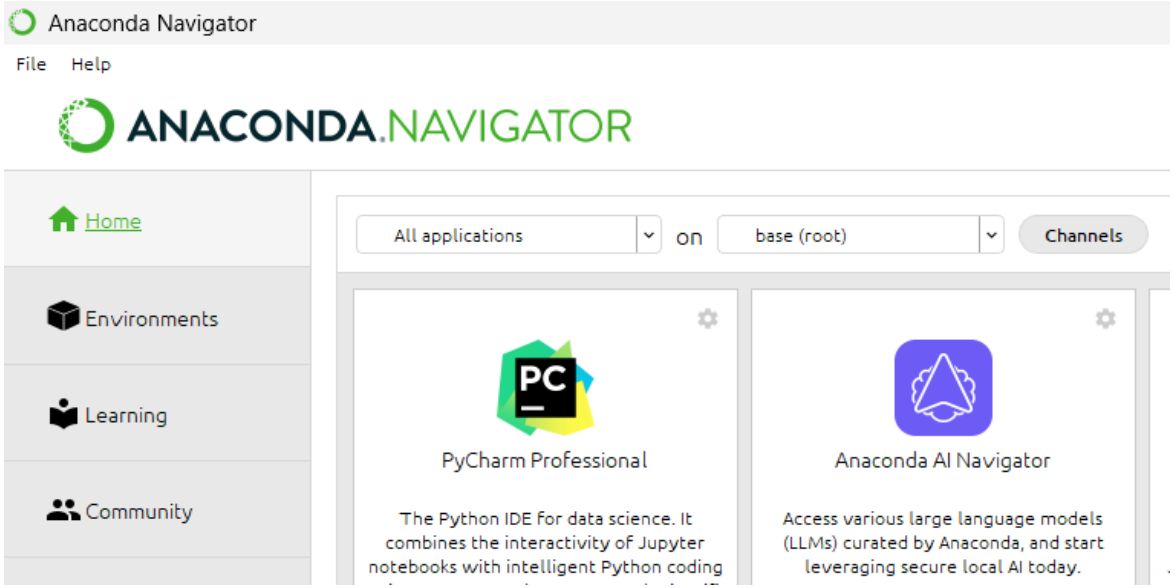
  - la gestion des environnements

  - Anaconda Navigator dans son ensemble

Anaconda : Gestion des environnements virtuels

📌 **Pourquoi utiliser un environnement virtuel ?**

Un environnement virtuel permet d'isoler les dépendances de chaque projet pour éviter les conflits entre différentes versions de bibliothèques.



Jupter Lab pour coder facilement et rapidement

- Créé en 2018 pour succéder à Jupyter Notebook
- IDE conçue pour des projets de Data Science

Points forts de Jupyter Lab

- Interface flexible et modulaire
- Exécution interactive du code
- Extensibilité

Rappels sur le langage Python

- Pour suivre ce cours il est nécessaire d'avoir en tête les bases du langage Python
- Pas de panique, nous allons revoir ensemble les bases afin de pouvoir mieux appréhender les solutions proposées par les autres bibliothèques
- Direction Jupyter Lab pour les tutos et les exercices pratiques.

Module 3 : Découverte des bibliothèques

1. Retour sur les opportunités des bibliothèques Pandas et Numpy

2. Pratique instantanée des bibliothèques en question

3. Projets pour rendre les activités plus intéressantes

Découverte de la librairie Numpy par la pratique

- Tout va à présent se dérouler dans les Notebooks et sur Jupyer Lab :
  - Présentation des possibilités de Numpy
  - Tuto pour voir ensemble les opportunités que nous offre la librairie
  - Exercices pour ancrer les compétences

Ressources Numpy

- https://numpy.org/
- https://datascientest.com/numpy
- https://github.com/numpy/numpy

Découverte de la librairie pandas par la pratique

- On continu sur Jupyter Lab :
  - Présentation des possibilités de Pandas
  - Tuto pour voir ensemble les opportunités que nous offre la librairie
  - Exercices pour ancrer les compétences

Ressources Pandas

- https://pandas.pydata.org/
- https://pypi.org/project/pandas/
- https://github.com/pandas-dev/pandas

Module 4 : L'analyse de données

- Objectifs principaux :
  - Comprendre la structure des données
  - Identifier les tendances
  - Détecter les valeurs aberrantes et les erreurs
  - Réaliser des prévisions

Objectif global : Savoir ce qu'on fait lorsque l'on manipule des données

Module 4 : L'analyse de données

1. Définition de l'analyse de données et les bases

2. Les différentes étapes

3. Découvertes des librairies de visualisation de données

Définition de l'analyse de données

- Définition : "L'analyse de données est un vaste domaine qui a pour objectif le traitement des données brutes en vue d'en tirer des informations nouvelles et utiles pouvant contribuer à transformer les entreprises, à accélérer l'innovation et à prévoir les résultats futurs."
- La science de l'analyse de données peut aider les entreprises et les organisations à comprendre leur passé et à orienter, voire à déterminer, leur avenir. En d'autres termes, une bonne analytique de données peut faire la différence entre succès et opportunités manquées.

Analyse de données et éthique

- L'analyse de données doit être réalisée de manière responsable pour garantir la transparence, la confidentialité et l'équité. Elle doit refléter la réalité des données.
- Principaux enjeux :
  - Biais des données
  - Confidentialité : Respect du RGPD
  - Transparence : Documenter et expliquer les méthodes d'analyses et les modèles utilisés
  - Impact social : Anticiper les conséquences des conclusions tirées des analyses

Exemple d'analyse de données biaisée

- **Contexte :** Une entreprise souhaite analyser les salaires de ses employés pour évaluer les inégalités éventuelles. Elle calcule le salaire moyen des hommes et des femmes travaillant dans l'organisation.
- **Problème :** L'analyse montre que le salaire moyen des hommes est plus élevé que celui des femmes. L'entreprise pourrait conclure à une inégalité salariale et envisager des ajustements.
- Cependant, **cette conclusion peut être biaisée** si d'autres facteurs ne sont pas pris en compte :
  - **Répartition des postes** : Si les hommes occupent davantage de postes à responsabilité
  - **Ancienneté** : Si les hommes sont employés depuis plus longtemps en moyenne, leurs salaires seront naturellement plus élevés.
  - **Secteurs d'activité** : Si l'entreprise emploie plus de femmes dans des départements moins rémunérateurs

- **Conséquences :** Une mauvaise interprétation des données pourrait conduire à **des décisions incorrectes**, comme ajuster les salaires de manière indiscriminée sans traiter la vraie cause du déséquilibre.
- **Solution :** Plutôt que de comparer uniquement les moyennes globales, il serait plus pertinent de :
  - **Segmenter l'analyse** par postes, ancienneté, ou département pour voir si l'écart persiste dans des conditions comparables.
  - **Utiliser la médiane** au lieu de la moyenne, afin d'éviter l'influence de valeurs extrêmes.
  - **Vérifier les écarts de salaires à poste et expérience égale**, pour s'assurer que l'inégalité ne provient pas d'une mauvaise répartition des employés.
- Cet exemple montre comment une **analyse descriptive mal conduite** peut induire des conclusions erronées si on ne prend pas en compte le contexte des données.

Figure 4a - Distribution des salaires mensuels nets en équivalent temps plein (EQTP) en 2019

	2019 (en euros)	Évolution entre 2018 et 2019 (en %, en euros constants)
1er décile	1 319	1,7
2e décile	1 469	2,1
3e décile	1 606	2,3
4e décile	1 759	2,5
Médiane	**1 940**	**2,6**
6e décile	2 165	2,6
7e décile	2 468	2,4
8e décile	2 929	1,7
9e décile	3 844	0,7
95e centile	4 982	– 0,1
99e centile	9 103	– 1,8
Moyenne	**2 424**	**1,2**

Lecture : en 2019, 10 % des salariés du privé et des entreprises publiques perçoivent un salaire mensuel net en EQTP supérieur à 3 844 euros (9e décile, D9).

Champ : France hors Mayotte, salariés en EQTP du privé et des entreprises publiques, y compris bénéficiaires de contrats aidés et de contrats de professionnalisation ; hors apprentis, stagiaires, salariés agricoles et salariés des particuliers employeurs.

Source : Insee, base Tous salariés 2019.

Btech Academy

Un exemple réel chez Amazon

- **Contexte :** En 2014, Amazon a développé un algorithme de recrutement basé sur l'analyse des CV des candidats pour automatiser la sélection des meilleurs profils. L'algorithme était entraîné sur les données des recrutements passés de l'entreprise, qui reflétaient les tendances historiques des embauches.
- **Problème :** L'algorithme a appris à favoriser les candidats masculins parce que les données d'entraînement étaient biaisées :
  - Amazon, comme beaucoup d'entreprises tech, avait historiquement embauché beaucoup plus d'hommes que de femmes pour des postes techniques.
  - L'algorithme a donc associé des mots-clés et des caractéristiques plus souvent présentes dans les CV masculins à un score plus élevé.
  - À l'inverse, il a commencé à pénaliser les CV mentionnant des termes comme "women's club" ou "female sports team", considérant implicitement ces candidatures comme moins adaptées.

- **Conséquences :**
  - L'algorithme a systématiquement désavantagé les candidates féminines, même lorsqu'elles avaient des compétences similaires aux hommes.
  - Cette erreur n'était pas intentionnelle, mais elle a montré comment un biais dans les données d'entraînement peut se traduire par un algorithme discriminatoire.
  - Amazon a finalement abandonné cet outil en 2018, reconnaissant qu'il n'était pas possible de garantir son impartialité.
- **Leçons à retenir :**
  - Les algorithmes ne sont pas neutres : ils reproduisent et amplifient les biais présents dans les données d'entraînement.
  - Une analyse de données doit être auditée pour détecter et corriger d'éventuelles discriminations avant d'être utilisée en prise de décision.
  - Corriger un biais ne signifie pas simplement supprimer une variable (ex. supprimer la mention du genre dans le CV) mais repenser l'ensemble du processus de collecte et d'interprétation des données.
- Ce cas montre comment **une analyse biaisée des données peut avoir un impact concret sur la vie des gens**, et pourquoi l'éthique est essentielle en data science.

Analyse de données et bonnes pratiques

- Analyser les données avec un regard critique
- Utiliser des méthodes d'audit et de validation des modèles
- Adopter une approche responsable et éthique dans la prise de décisions

Les grandes étapes de l'analyse de données

1. Acquisition des données
2. Nettoyage des données
3. Exploration des données
4. Modélisation et Analyse
5. Visualisation et interprétation
6. Communication et prise de décision

Acquisition des données

- Rassembler/agréger les données pertinentes. Cela peut inclure :
  - Des fichiers CSV, Excel, JSON…
  - Des bases de données relationnelles (SQL) ou NoSQL
  - Des API et du Web Scraping
  - Des capteurs IoT ou des logs d'applications

Nettoyage des données

- Les données brutes sont souvent incomplètes, erronées ou incohérentes. Cette étape consiste à :
  - Gérer les valeurs manquantes (suppression, imputation par la moyenne, etc.)
  - Corriger les erreurs (dates incorrectes, fautes de frappe, doublons)
  - Uniformiser les formats (standardiser les unités, les types de données)

  Exemple : Remplacer les valeurs manquantes du revenu moyen par la médiane de l'échantillon

Exploration de données

- Cette étape permet de comprendre la structure des données et d'identifier les tendances principales. Elle inclut :
  - Calcul des statistiques descriptives (moyenne, médiane, écart-type)
  - Détection des valeurs aberrantes
  - Visualisation préliminaire (histogrammes, boxplots, heatmaps)
  - Transformation des variables (normalisation, encoding des variables catégorielles)

  Exemple : Observer la répartition des salaires par secteur avec un boxplot.

Exploration de données - L'exemple du Clustering

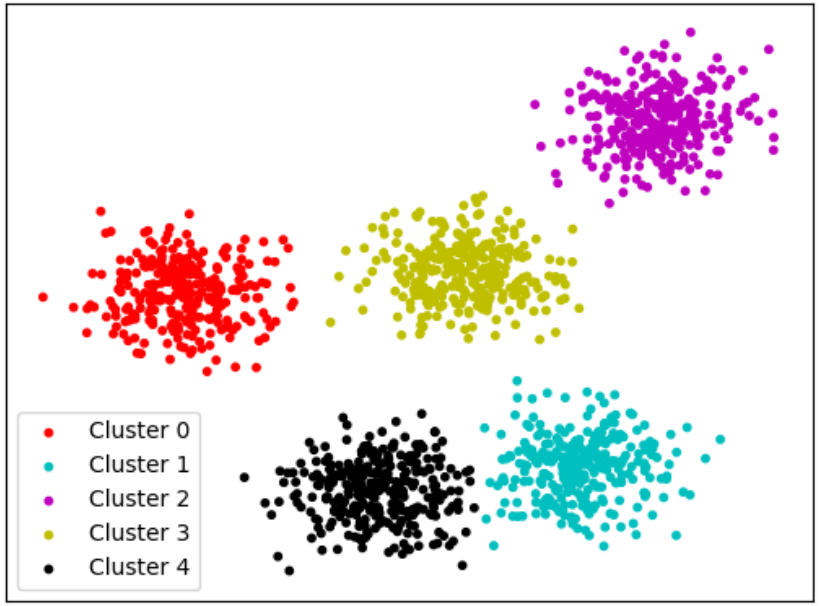
- Explications sur le clustering :
  - Regroupement de données en fonction de leur similarité (les objets similaires sont regroupés dans le même "cluster" tandis que les objets dissemblables se retrouvent dans des clusters différents)
- **Objectif principal** : Découvrir des structures ou des regroupements cachés dans les données sans utiliser de labels.
- **Applications** :

Segmentation de clients

Analyse de comportements d'achat

Vision par ordinateur (ex. détection d'objets similaires)

Recommandations de produits



Modélisation et analyse

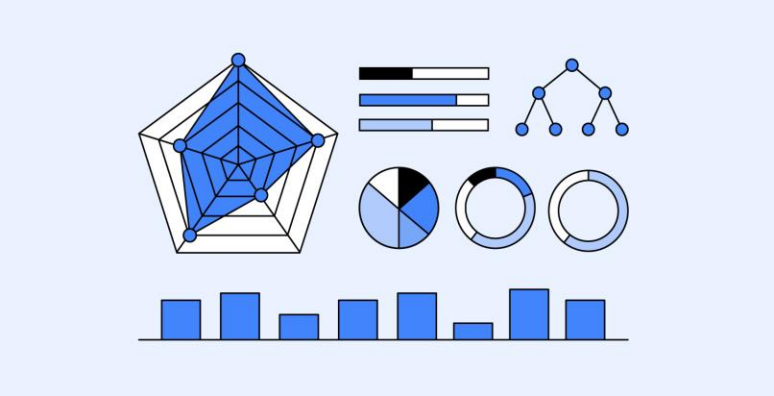
- A ce stade, on extrait des informations exploitables via :
  - L'analyse descriptive : Tendances générales, corrélations, distributions
  - L'analyse diagnostique : Identifier les relations causales
  - L'analyse prédictive : Prédiction des ventes futures avec des modèles statistiques ou de Machine Learning
  - L'analyse prescriptive : Recommandation d'actions basées sur les résultats

Exemple : Détecter une corrélation entre l'âge et la consommation d'un produit.

Visualisation et interprétation

- Les résultats doivent être présentés de manière compréhensibles via :
  - Des graphiques et dashboards (Matplotlib, Seaborn, Power BI, Tableau)
  - Des rapports analytiques (résumé des tendances et recommandations)
  - Des présentations claires pour les parties prenantes

Exemple : Présenter une évolution des ventes mensuelles sous forme de courbe pour anticiper la saisonnalité.



Communication et prise de décision

- L'analyse des données doit servir à la prise de décision. A cette étape, on :
  - Fournit des recommandations basées sur les insights
  - Identifie des actions concrètes à mettre en place
  - Evalue les limites de l'analyse (biais, fiabilité des données)

Exemple : recommander d'augmenter les stocks d'un produit en prévision d'une forte demande détectée par l'analyse.

Pour conclure sur l'analyse de données

- L'analyse de données ne se limite pas à un simple traitement statistique. Elle suit un processus structuré, depuis la collecte jusqu'à l'interprétation, afin d'aboutir à des décisions éclairées.

Découvertes des bibliothèques de visualisation

- 📊 **De l'Analyse à la Visualisation**
- **Pourquoi la visualisation est essentielle ?**
- ✔️ Facilite l'interprétation des données complexes
- ✔️ Met en évidence les tendances, corrélations et anomalies
- ✔️ Améliore la prise de décision grâce à des représentations claires
- **Ce que nous allons voir ensuite :**
- 🔹 **Matplotlib** : La base de la visualisation en Python
- 🔹 **Seaborn** : Pour des graphes plus stylés et analytiques
- 🔹 **Cas pratiques** : Création et personnalisation de graphiques
- 🎯 **Objectif : Transformer nos analyses en insights visuels !**

Découverte de Matplotlib par la pratique

- Présentation des possibilités de matplotlib
- Tuto pour voir ensemble les opportunités que nous offre la librairie
- Exercices pour ancrer les compétences

Ressources Matplotlib

- https://matplotlib.org/
- https://fr.matplotlib.net/stable/tutorials/index.html#google_vignette
- https://github.com/matplotlib/matplotlib

Découverte de Seaborn par la pratique

- Présentation des possibilités de Seaborn
- Tuto pour voir ensemble les opportunités que nous offre la librairie
- Exercices pour ancrer les compétences

Ressources Seaborn

- https://seaborn.pydata.org/
- https://pypi.org/project/seaborn/
- https://github.com/mwaskom/seaborn
- https://seaborn.pydata.org/tutorial/introduction.html

Manipulation d'images

- Modifier / analyser / transformer des images numériques avec des algorithmes.
- Les principales bibliothèques en Python :
  - **PIL/Pillow** : Chargement, transformation et sauvegarde d'images
  - **OpenCV** : Traitement avancé des images et vision par ordinateur
  - **Matplotlib** : Affichage et analyse des images
  - **Scikit-image** : Analyse scientifique et transformations avancées

Cas d'utilisation concrets

✅ **Retouche et amélioration d'images** (ex : Instagram, Photoshop)

✅ **Reconnaissance faciale et détection d'objets** (ex : caméras de surveillance intelligentes)

✅ **OCR (Reconnaissance de texte dans les images)** (ex : Google Lens, Adobe Scan)

✅ **Imagerie médicale et analyse biomédicale** (ex : détection de tumeurs sur des IRM)

✅ **Génération et transformation d'images par IA** (ex : filtres artistiques, deepfake)

Exemple de projet réel - Facebook DeepFace

📌 🔍 **Présentation du Projet**

DeepFace est un projet développé par Facebook utilisant Python et OpenCV pour la reconnaissance faciale avancée. Il exploite des réseaux de neurones profonds pour identifier et vérifier les visages avec une précision quasi-humaine.

📌 🚀 **Technologie utilisée :**

- **OpenCV** : Traitement des images et détection des visages
- **Deep Learning (TensorFlow, Keras)** : Modélisation et reconnaissance faciale
- **Pandas & NumPy** : Gestion des données et optimisation des calculs

Exemple de projet réel - Facebook DeepFace

📌 🌍 **Cas d'utilisation :**

- 📸 **Tag automatique des photos** sur Facebook
- 🔒 **Sécurité biométrique** (ex : déverrouillage par reconnaissance faciale)
- 🏛️ **Surveillance intelligente** (ex : caméras de sécurité intelligentes)
- 🔗 **Impact :** Ce projet a révolutionné la reconnaissance faciale en permettant une identification rapide et précise, influençant de nombreux secteurs comme la sécurité et les réseaux sociaux.

Découverte de OpenCV par la pratique

- Présentation des possibilités de OpenCV
- Tuto pour voir ensemble les opportunités que nous offre la librairie
- Exercices pour ancrer les compétences

Ressources OpenCV

- https://opencv.org/
- https://github.com/opencv/opencv

Les gros volumes de données

- 📌 **Pourquoi est-ce un défi ?**
  Le big data implique de traiter des ensembles de données volumineux qui ne tiennent pas en mémoire et nécessitent des solutions optimisées pour le stockage et le calcul.
- 🔄 **Stratégies pour Optimiser le Traitement**
- 🔹 **Échantillonner les données** avant traitement.
  🔹 **Utiliser des bases de données optimisées** plutôt que de tout stocker en fichiers CSV.
  🔹 **Traitement en streaming** plutôt que de charger tout en RAM.
  🔹 **Exploiter le cloud** (Google BigQuery, AWS S3, Azure Data Lake).
- 👉 **Avec les bons outils, gérer des téraoctets de données devient possible !** 🚀

Exemple réel : Uber

- 📌 🚕 **Cas d'utilisation : Uber et l'analyse des trajets en temps réel**
  Uber traite des millions de trajets chaque jour. L'entreprise utilise PySpark pour analyser ces données en temps réel et optimiser son service.
- 🚀 **Technologies utilisées**
- ✅ **PySpark** : Traitement distribué sur plusieurs machines
  ✅ **Hadoop HDFS** : Stockage efficace des logs et trajets
  ✅ **Kafka** : Streaming des données en temps réel
  ✅ **AWS S3 / Google BigQuery** : Stockage cloud pour la scalabilité
  ✅ **Machine Learning** : Optimisation des prix et des itinéraires

Exemple réel : Uber

📍 **Optimisation des itinéraires** 🚚 → Réduction du temps de trajet
💰 **Dynamic Pricing (Tarification en temps réel)** 📊 → Ajustement des prix selon la demande
🔍 **Détection de fraudes** 🔐 → Identification des courses suspectes
📈 **Analyse des tendances** 📉 → Prédiction de la demande en fonction du climat, des événements

🌍 **Impact de PySpark chez Uber :**

🚀 **+100 millions de trajets traités par jour**
📉 **Réduction des coûts de calcul** grâce au traitement distribué
⚡ **Optimisation du temps de réponse** pour l'application mobile

👉 **PySpark est une solution clé pour traiter des volumes massifs de données en temps réel !** 🔥

Découverte de PySpark par la pratique

- Présentation des possibilités de PySpark
- Tuto pour voir ensemble les opportunités que nous offre la librairie
- Exercices pour ancrer les compétences

Ressources PySpark

- https://spark.apache.org/docs/latest/api/python/index.html
- https://medium.com/towards-data-engineering/uber-data-analysis-project-in-pyspark-e105a445fc3e

Manipulation de textes

- Introduction au traitement du texte
- Librairies incontournables
- Applications et utilité

Qu'est-ce que la manipulation de texte

- **Définition** : La manipulation de texte consiste à effectuer des opérations sur des chaînes de caractères pour extraire, modifier, analyser ou transformer le contenu.
- **Exemples d'opérations courantes** :
  - Recherche et remplacement
  - Découpage et fusion de textes
  - Analyse de fréquence
  - Nettoyage et prétraitement de données textuelles

Librairies incontournables pour la manipulation de texte

- RE (expression régulières)
- NLTK (Natural Language Toolkit)
- Spacy

Les utilités du traitement de texte

- La manipulation de texte en Python est essentielle dans de nombreux domaines comme le **traitement du langage naturel (NLP)**, l'**analyse de données** et l'**extraction d'informations**.
- Des **librairies** comme `re`, `nltk`, `pandas`, ou `textblob` offrent des outils puissants et simples pour travailler avec du texte.
- Le traitement du texte est crucial pour des applications comme **l'analyse de sentiment**, la **recherche d'information**, et le **nettoyage des données**.

Ce que vous avez appris en suivant ce cours

- Les différentes étapes de manipulations de données
- Les bibliothèques Python essentielles pour la manipulation de données
- Réaliser des analyses et visualisations de données de cas concrets à travers différents exercices et projets

Merci d'avoir pris part à ce cours

Toutes les ressources qui ont contribué à l'élaboration de ce cours

- https://siecledigital.fr/2019/01/15/carrefour-veut-reduire-le-gaspillage-alimentaire-grace-a-lintelligence-artificielle/
- https://gist.github.com/nalron/
- https://www.kaggle.com/code/barabaradiop/python-pour-la-science-des-donn-es
- https://www.youtube.com/@foxxpy
- https://github.com/foxxpy
- https://chatgpt.com/

Suite des ressources

- https://www.hpe.com/fr/fr/what-is/data-analytics.html#:~:text=L'analyse%20de%20donn%C3%A9es%20est,%C3%A0%20pr%C3%A9voir%20les%20r%C3%A9sultats%20futurs.
- https://www.lefigaro.fr/social/2018/10/11/20011-20181011ARTFIG00096-le-logiciel-de-recrutement-d-amazon-n-aimait-pas-les-femmes.php
- https://www.lavoixdunord.fr/1442978/article/2024-03-21/comprendre-la-difference-entre-salaire-moyen-et-salaire-median

Suite des ressources

- https://github.com/JosueAfouda/TUTORIALS
- https://www.amazon.fr/stores/author/B08F17S1V8/allbooks
- https://www.youtube.com/channel/UCpd56FfjlkKbkHlbgY6XE3w
- https://github.com/handee/opencv-gettingstarted/tree/master
- https://github.com/awesomedata/awesome-public-datasets
- https://donnees.banquemondiale.org/indicateur/SP.DYN.LE00.IN
- https://www.beedeez.com/fr/guides-ultimes/social-learning

Rappels matrices

Définition : Une **matrice** est un tableau de nombres (ou d'autres objets mathématiques) organisés en lignes et colonnes. Elle est souvent utilisée en algèbre linéaire, notamment dans la résolution de systèmes d'équations linéaires, la transformation géométrique, ou l'apprentissage machine.

Notation d'une matrice :

- Une matrice de dimension $m \times n$ (m lignes et n colonnes) est notée $A = [a_{ij}]$ où :
  - i est l'indice de la ligne.
  - j est l'indice de la colonne.

Exemple de matrice 3×2 :

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

Matrices – les opérations de base

$$\begin{bmatrix} 1 & 2 & 0 \\ 4 & 3 & -1 \end{bmatrix} + \begin{bmatrix} 5 & 2 & 3 \\ 1 & 3 & 4 \end{bmatrix} = \begin{bmatrix} 6 & 4 & 3 \\ 5 & 6 & 3 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 0 \\ 4 & 3 & -1 \end{bmatrix} - \begin{bmatrix} 5 & 2 & 3 \\ 1 & 3 & 4 \end{bmatrix} = \begin{bmatrix} -4 & 0 & -3 \\ 3 & 0 & -5 \end{bmatrix}$$

Matrices - multiplication

$$[A]$$

$$[A^2] \quad [A^3]$$

$$\begin{pmatrix} 1 & 0 & 3 \\ 0 & 3 & 1 \\ 3 & 1 & 0 \end{pmatrix}$$

$$\begin{pmatrix} 10 & 3 & 3 \\ 3 & 10 & 3 \\ 3 & 3 & 10 \end{pmatrix} \quad \begin{pmatrix} 10\times1+3\times3 & 3\times3+3\times1 & 3\times10+3\times1 \\ 3\times1+3\times3 & 10\times3+3\times1 & 3\times3+10\times1 \\ 3\times1+3\times10 & 3\times3+10\times1 & 3\times3+3\times1 \end{pmatrix}$$

Rappels - algèbre linéaire

Définition : L'**algèbre linéaire** est une branche des mathématiques qui étudie les **espaces vectoriels** et les **applications linéaires** entre ces espaces. Elle est fondamentale dans de nombreux domaines, notamment en informatique, en physique, et en ingénierie.

Concepts clés :

1. **Vecteurs :** Un vecteur est un objet mathématique qui peut être vu comme une liste de nombres (composantes) représentant une direction et une magnitude dans un espace donné.
   - Exemple : $\vec{v} = (v_1, v_2, v_3)$ dans un espace à 3 dimensions.
2. **Espaces vectoriels :** Un ensemble de vecteurs où l'on peut effectuer des opérations comme l'addition et la multiplication par un scalaire.
3. **Matrices :** Une matrice représente une transformation linéaire qui agit sur des vecteurs. Les matrices permettent de décrire des systèmes d'équations linéaires ou des transformations géométriques.
4. **Systèmes d'équations linéaires :** Un ensemble d'équations qui peut être résolu en utilisant des vecteurs et des matrices, souvent par des méthodes comme l'élimination de Gauss ou la décomposition LU.

Rappels - algèbre linéaire

- **1. Résolution de Systèmes Linéaires :**
  L'algèbre linéaire permet de résoudre des systèmes d'équations linéaires sous forme matricielle, ce qui est crucial pour l'analyse de données, l'optimisation, et les simulations numériques.
- **2. Transformations Géométriques :**
  Les matrices sont utilisées pour appliquer des transformations géométriques comme la translation, la rotation, la mise à l'échelle dans des graphiques 3D ou des applications de vision par ordinateur.
- **3. Apprentissage Automatique :**
  Les concepts d'algèbre linéaire sont au cœur de nombreuses techniques d'apprentissage automatique, telles que les régressions linéaires, les réseaux neuronaux et les réductions de dimensions (PCA).
- **4. Graphes et Réseaux :**
  Les matrices d'adjacence et les matrices de transition sont utilisées pour modéliser et analyser des graphes dans les domaines des réseaux sociaux, des systèmes de recommandation et des réseaux de communication.

Rappels : Régression linéaire

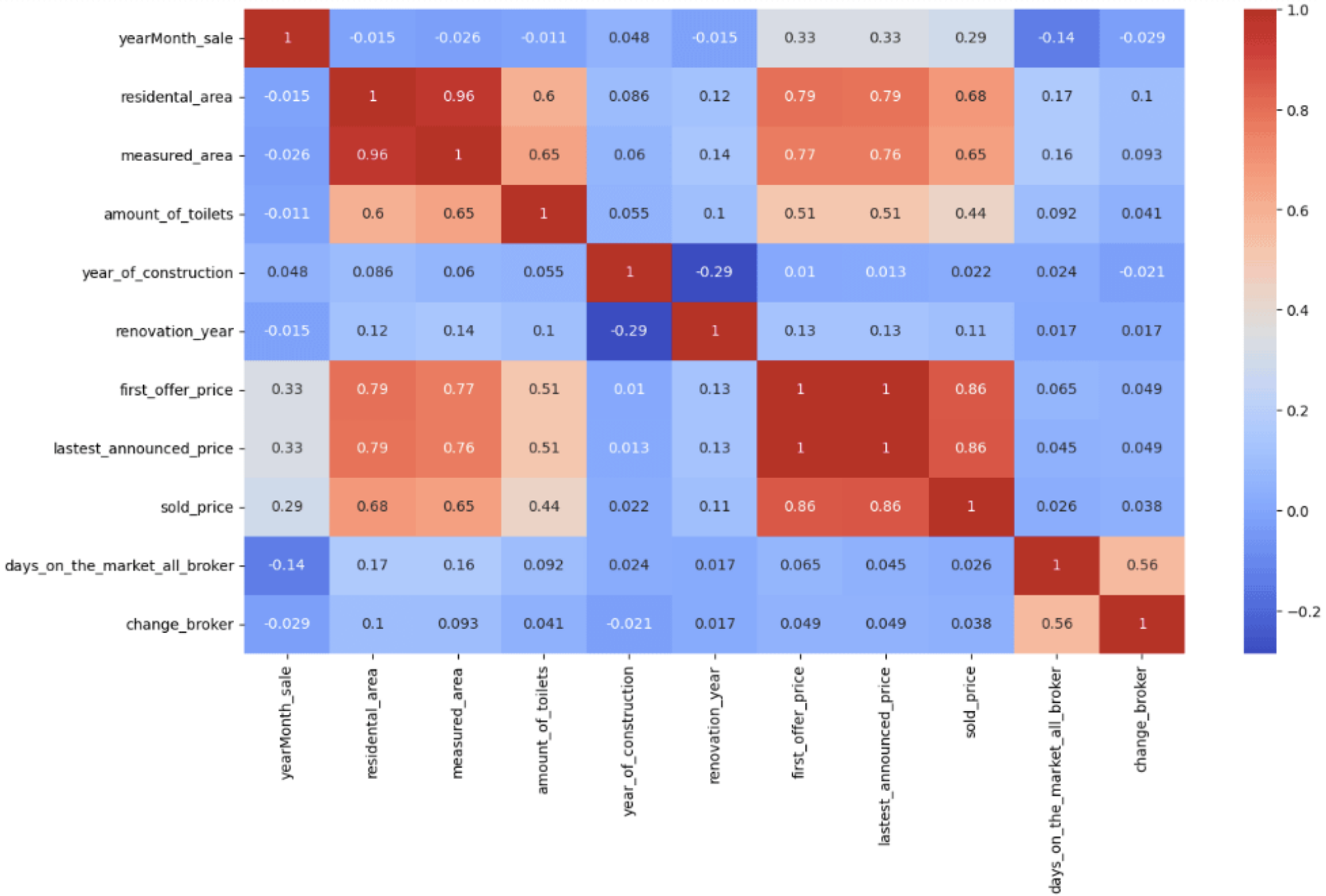
- **Objectif** : Modéliser la relation entre une variable dépendante y et une ou plusieurs variables indépendantes x.
- **Formule** :
  $y = \beta_0 + \beta_1 x$
- **Méthode** : Estimer les coefficients β_0 et β_1 pour minimiser l'erreur (SSE).
- **Évaluation** :
  - R^2 : Proportion de variance expliquée par le modèle.
- **Applications** : Prédiction (prix, ventes, etc.).

Rappels : Corrélation linéaire

- **Définition** : Mesure de la force et de la direction de la relation linéaire entre deux variables.
- **Coefficient de corrélation de Pearson** r :
  - **Formule** :
    $$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}}$$
  - **Plage de valeurs** :
    - $r = 1$: Corrélation parfaitement positive.
    - $r = -1$: Corrélation parfaitement négative.
    - $r = 0$: Pas de corrélation linéaire.
- **Interprétation** :
  - **Positif** : Les variables augmentent ensemble.
  - **Négatif** : L'une augmente pendant que l'autre diminue.
- **Applications** : Analyser la relation entre deux variables (ex. : taille et poids, température et ventes).

Rappels - Matrice de Corrélation

- **Qu'est-ce que c'est ?**
  Une matrice carrée qui montre les relations entre plusieurs variables.
- **Comment elle fonctionne ?**
  Chaque case représente la corrélation entre deux variables (de -1 à 1).
  (Exemple d'une matrice de corrélation en slide 99.)
- **Interprétation des valeurs :**
- **1** : Corrélation parfaite positive
- **-1** : Corrélation parfaite négative
- **0** : Aucune corrélation



Btech
Academy

Btech
Academy